Proactive Budgeting

Execution Plan & Architectural Design

Team:

- Bank Bálint Dániel (V90WCP)
- Géczy Krisztián (MF5M7S)
- Mocsi Lilla (MF9RFV)
- Tamás Bence (BVVZYV)
- Tóth Kiara Valentina (MJR7PO)

1. Architectural Design

This section defines the technical structure of the Proactive Budgeting application: the technology choices made, how the system components interact, how data flows through the system, and how the application is deployed. These decisions were made to serve the validated MVP user story set within a five-week development window with a five-person team.

1.1 Technology Stack

The stack was chosen to balance developer familiarity, ecosystem maturity, and deployment simplicity. Each layer is independently testable and can be developed in parallel by separate team members.

Frontend: React 19 with TypeScript, built by Vite, styled with Tailwind CSS

Backend: Python 3.12 with FastAPI, served by Uvicorn, with APScheduler for in-process background jobs

Database: PostgreSQL 16 with a named Docker volume for data persistence

Auth: JWT-based authentication implemented in FastAPI using python-jose and passlib. Access tokens are short-lived (30 min); refresh tokens are stored server-side.

Reverse Proxy: Nginx routes all traffic: /api/* is proxied to the FastAPI backend; all other paths serve the static Vite build. TLS termination is handled at the Nginx layer.

Containerisation: Docker Compose orchestrates four containers on a single VPS: nginx, frontend (static build), backend, and db. A shared internal Docker network connects backend and db; only ports 80 and 443 are exposed to the internet.

Version Control: Git with GitHub

1.2 System Component Overview

The system is composed of five logical components. The diagram in Appendix A illustrates their relationships.

- **React Frontend:** React Frontend (SPA): The single-page application runs entirely in the browser. It communicates with the backend exclusively via the REST API using JSON over HTTPS. There is no server-side rendering. The Vite build output is a set of static files served by Nginx.
- **FastAPI Backend:** FastAPI Backend: The backend exposes a versioned REST API under /api/v1/. It handles authentication, all business logic (rule engine, spending-pace algorithm, alert generation), and database access via SQLAlchemy ORM. APScheduler runs inside the same backend process and executes the spending-pace check job on a scheduled interval.
- **APScheduler:** APScheduler (in-process): A background job scheduler embedded in the FastAPI process. It runs two recurring jobs: the spending-pace evaluator (checks every 6 hours whether any user's category is trending over budget and writes alert records to the database) and the event reminder job (runs daily and checks whether any upcoming event falls within the 14-28 day notification window). Alerts are stored as records in the database and fetched by the frontend on the next dashboard load.
- **PostgreSQL:** PostgreSQL Database: The single source of truth for all persistent data. Tables include users, budget_rules, transactions, categories, events, and alerts. The database runs in its own container with a named volume so that data survives container restarts. The backend is the only component that talks directly to the database.

- **Nginx:** Nginx Reverse Proxy: The only internet-facing component. It terminates TLS, routes /api/* requests to the FastAPI container on its internal port, and serves the static frontend files for all other routes. This means the frontend and backend share a single domain and port, eliminating CORS complexity in production.

1.3 Authentication Design

Authentication is implemented entirely within the FastAPI backend. On registration, the user's password is hashed using bcrypt via passlib and stored in the users table. On login, the backend validates the credentials, generates a signed JWT access token (30-minute expiry) and a refresh token (7-day expiry). The refresh token is stored in the database against the user record to enable server-side invalidation. The frontend stores the access token in memory (not localStorage) and attaches it as a Bearer token in the Authorization header of every API request. When the access token expires, the frontend uses the refresh token to obtain a new one transparently. All protected API routes use a FastAPI dependency that validates the token signature and expiry on every request.

1.4 Spending-Pace Algorithm

The proactive alert system (US-05, US-07, US-08) is built on a single spending-pace algorithm that runs in the background scheduler. The algorithm operates as follows:

- For each user with an active budget, retrieve all transactions logged in the current calendar month.
- Group transactions by category and sum the amounts to get the current spend per category.
- Calculate the spend rate: current spend divided by the number of elapsed days in the month.
- Project the spend rate forward to the end of the month: spend rate multiplied by total days in the month.
- Add any known upcoming event costs (from US-09) that fall within the current month to the projection.
- Compare the projected total against the category budget limit derived from the user's percentage rules and monthly income.
- If the projection exceeds the limit, write an alert record to the alerts table with type BREACH_PREDICTED.
- If the projection is within limit but the category has already consumed more than its proportional share for the elapsed days (i.e. pace is ahead of schedule), write an alert with type PACE_ELEVATED. This is the early-in-month signal for US-07.

The frontend fetches unread alerts from GET /api/v1/alerts on every dashboard load and marks them as read after display. No push infrastructure is required.

1.5 Data Model

The database schema is organised around six core tables. Relationships are as follows:

- users: id, email, password_hash, created_at, refresh_token
- budget_rules: id, user_id (FK), label, percentage, monthly_income, currency (HUF/EUR), updated_at
- categories: id, user_id (FK), name, icon, is_savings_goal, monthly_target
- transactions: id, user_id (FK), category_id (FK), amount, currency, description, transaction_date, created_at
- events: id, user_id (FK), name, estimated_cost, currency, event_date, is_recurring, recurrence_months
- alerts: id, user_id (FK), category_id (FK, nullable), alert_type, message, is_read, created_at

Each user owns all their data rows directly. There are no shared data models at MVP. The shared household feature (US-20) is deferred to post-MVP and would require a household table and permission model not present in this schema.

1.6 Deployment Architecture

The application is deployed on a single VPS using Docker Compose. The deployment diagram in Appendix B shows the full container topology. The four containers and their roles are:

- **nginx:** nginx (port 80/443 exposed): Reverse proxy and TLS termination. Serves static frontend files and proxies /api/* to the backend container on port 8000 via the internal Docker network.
- **frontend build:** frontend: Contains the Vite production build output. In the Docker Compose setup the static files are copied into the nginx container at build time using a multi-stage Dockerfile, so this is not a separate running container but a build stage.
- **backend:** backend (internal port 8000): FastAPI application served by Uvicorn with four workers. APScheduler runs in the same process. Connects to the db container on port 5432 via the internal Docker network. The DATABASE_URL, JWT_SECRET, and other secrets are injected as environment variables from a .env file on the VPS, never committed to the repository.
- **db:** db (internal port 5432): PostgreSQL 16. Data is stored in a named Docker volume (postgres_data) that persists across container restarts and redeployments. The volume is backed up by a daily cron job on the VPS that copies a pg_dump to a separate directory.

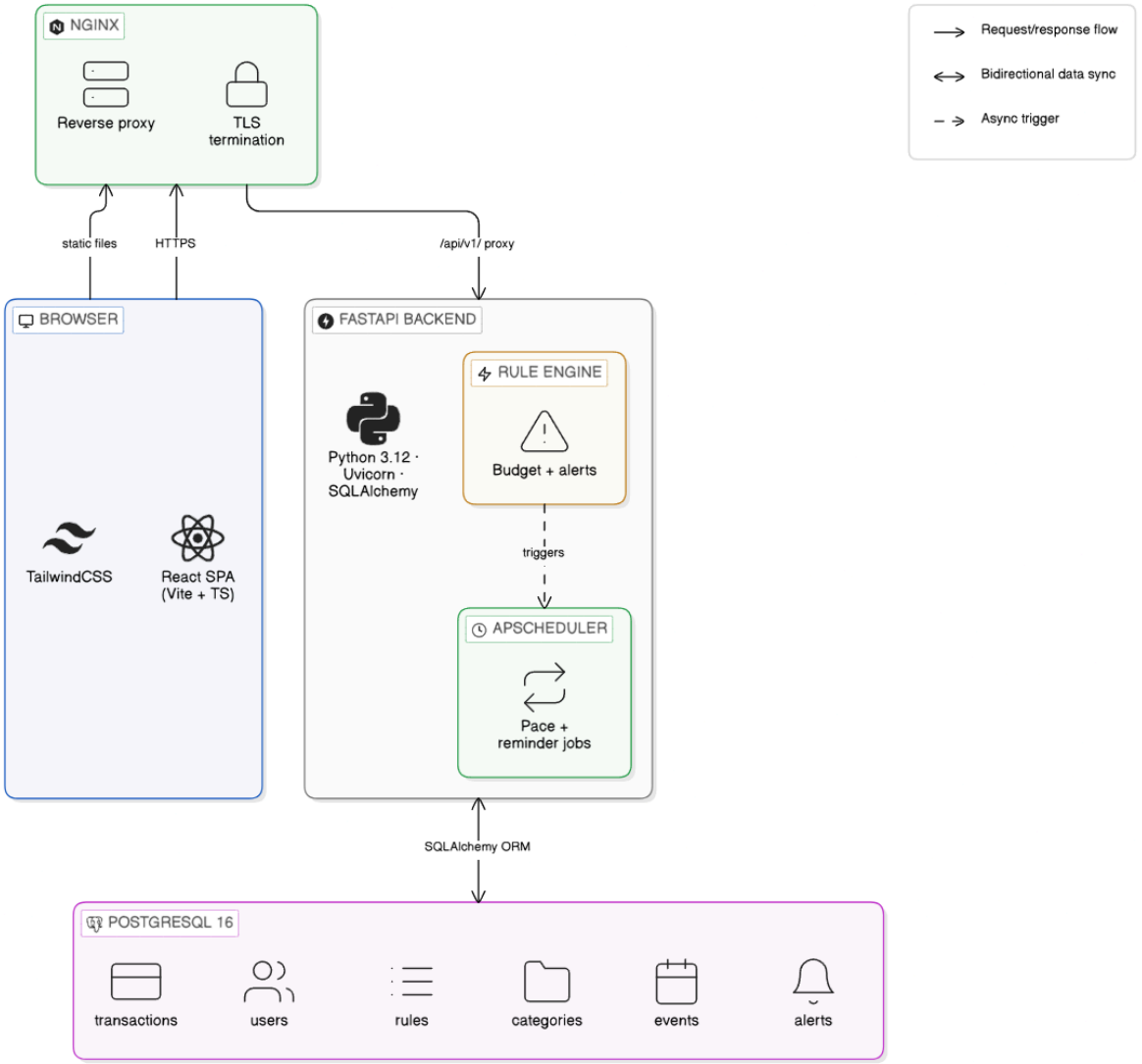
Deployment is triggered manually by SSH-ing into the VPS, pulling the latest main branch, and running docker compose up --build -d. A CI check on GitHub Actions runs the backend test suite on every pull request to main, but deployment is not automated at MVP stage.

1.7 API Design Conventions

The backend exposes a RESTful API under the /api/v1/ prefix. Key conventions are:

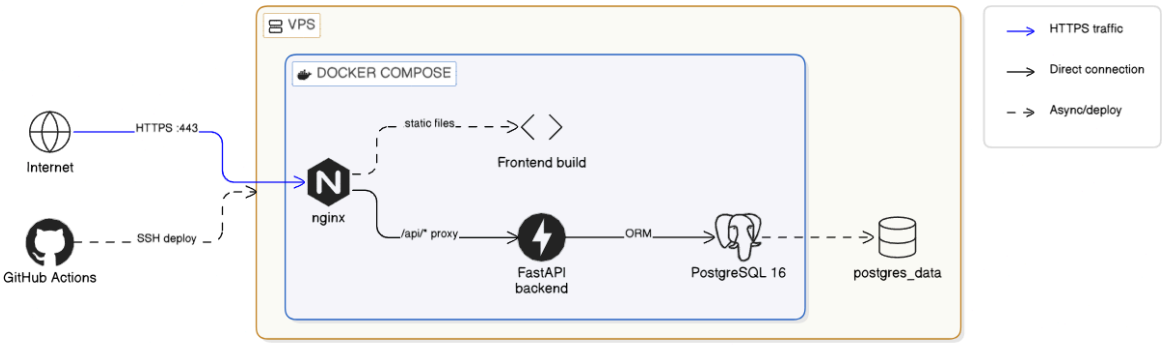
- All endpoints return JSON. HTTP status codes follow REST conventions: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 404 Not Found, 422 Unprocessable Entity.
- All protected endpoints require a valid JWT Bearer token in the Authorization header.
- Request and response bodies are validated using Pydantic v2 schemas, which also auto-generate the OpenAPI documentation available at /api/v1/docs.
- Dates and times are stored and returned as ISO 8601 strings in UTC.
- Currency amounts are stored as integers in the smallest currency unit (fillér for HUF, cent for EUR) to avoid floating-point rounding errors.

System Component Diagram



eraser

Deployment Diagram



eraser

2. Work Breakdown Structure

Each task is tied to a user story, estimated in working days, and assigned to a named developer. Tasks are sized to complete in 1–3 days. The done criteria state the concrete, testable conditions for accepting each task as complete.

Developer assignments

ID	US	Task & description	Done criteria	Who	Est.
AUTH prerequisite (all stories)					
T-01	AUTH	User registration & password hashing POST /auth/register; bcrypt hash via passlib; store user in DB.	201 on success; 422 on duplicate email; password never stored plaintext.	Bálint	1d
T-02	AUTH	JWT login, access & refresh tokens POST /auth/login; signed access token (30 min) + refresh token (7 days) stored in DB.	Both tokens returned on login; expired token rejected 401; refresh endpoint issues new access token.	Bence	1d
T-03	AUTH	Auth middleware & protected route dependency FastAPI Depends() validating JWT on every protected route; attaches user_id to request state.	Missing token → 401; tampered token → 401; valid token passes through to handler.	Bence	1d
T-04	AUTH	Login / register screens (frontend) React pages for register and login; form validation; store access token in memory; attach Bearer to all API calls.	User can register, log in, and reach dashboard; token attached to subsequent requests without page reload.	Lilla	1d
US-01, US-02, US-03 | Rule engine & income setup					

T-05	US-01/02/03	Budget rule data model & CRUD API budget_rules table (label, percentage, monthly_income, currency); POST/GET/PUT/DELETE /rules; validation: percentages must sum to 100.	CRUD persists correctly; sum ≠ 100 returns 422; all endpoints require auth.	Bálint	2d
T-06	US-01/02/03	Rule engine calculation service Python service: rules + income → per-category HUF/EUR amounts; recalculates on any rule change.	Unit tests pass for 60/20/20, 50/30/20, and custom splits; rounding error < 1 HUF.	Kiara	1d
T-07	US-01/02/03	Budget setup UI — rule input screen React screen: income field (HUF/EUR toggle), percentage inputs per category, live HUF amount display, save button.	Amounts update instantly on input change; saving persists rules; sum ≠ 100% blocks save with inline error.	Krisztián	2d
T-08	US-01/02/03	Category management UI Add, rename, delete budget categories; icon picker; savings goal toggle per category.	New category appears in rule setup and transaction entry; deleted category prompts transaction reassignment.	Lilla	2d
US-13 | Fast manual transaction entry					
T-09	US-13	Transaction data model & API transactions table (amount, currency, description, category_id, transaction_date); POST/GET/DELETE /transactions.	POST stores and returns 201; GET returns paginated list filterable by date and category; auth required.	Bálint	1d
T-10	US-13	Fast entry UI — spinner input & autocomplete Bottom-sheet modal: scroll-wheel amount input, category picker, optional description, date	Transaction logged in under 5 taps; spinner increments correctly; entry appears in list immediately after save.	Krisztián	3d

		defaults to today.			
US-05, US-07, US-08 | Proactive alerts & projections					
T-12	US-05/07/08	Spending-pace algorithm Python service: sum transactions per category, compute spend rate (spent ÷ elapsed days), project to month-end, add upcoming event costs.	Unit tests cover 5 scenarios (on-track, pace-elevated, breach-predicted, sparse data, zero spend); results within 1 HUF of manual calculation.	Kiara	2d
T-13	US-05/07/08	Alert generation & APScheduler job APScheduler job every 6 hours; calls pace service for every active user; writes BREACH_PREDICTED or PACE_ELEVATED alert records; GET /alerts endpoint.	Scheduler fires on startup and every 6 hours; alerts in DB within 6 hours of threshold breach; max 1 alert per category per day.	Bence	2d
T-14	US-05/07/08	Alert banner & projected balance UI Dashboard alert banner for unread alerts with category and message; per-category projected end-of-month balance on budget overview screen.	Alerts appear on dashboard load; dismiss marks as read; projected balances colour-coded red/amber/green by severity.	Krisztián	2d
US-06 | Daily safe-to-spend					
T-15	US-06	Safe-to-spend calculation & API endpoint GET /dashboard/safe-to-spend; formula: (remaining budget − upcoming event costs this month) ÷ remaining days.	Correct value returned for boundary cases: month start, month end, all budget spent, no income entered.	Bálint	1d
T-16	US-06	Safe-to-spend headline widget on dashboard Prominent hero number on main dashboard with currency, label, and colour indicator (green/amber/red).	Updates on each load; green >80%, amber 50–80%, red <50% of daily budget remaining.	Bálint	1d
US-09, US-10 | Event planning					

T-17	US-09/10	Event model & API events table (name, estimated_cost, event_date, category_id,); full CRUD; future-date validation.	POST creates event; GET returns events sorted by date	Kiara	2d
T-18	US-09/10	Event reminder APScheduler job Daily job: finds events with date between now+14 and now+28 days; writes EVENT_REMINDER alert if not already issued.	Reminder alert appears 14–28 days before event; not re-sent if already issued; visible in /alerts feed.	Kiara	1d
T-19	US-09/10	Event planner UI — add & manage events Screen to add named events with estimated cost, date, category; upcoming events list sorted by date.	Event form submits and appears in list; cost reflected in safe-to-spend	Lilla	2d
US-15 | Category spending history					
T-20	US-15	Category detail screen — transaction list & summary Tappable category opens detail screen: spending total, % used, scrollable transaction list for current month.	All transactions for selected category and month shown; totals match API; empty state shown with helpful copy.	Krisztián	2d
US-16, US-17 | Savings goals & progress					
T-21	US-16/17	Savings goal configuration & progress API categories table savings_goal flag and monthly_target field; GET /goals/progress returns contributed amount vs target for current month.	Goal progress percentage correct for 0%, 50%, and 100% contribution; resets at month start.	Kiara	1d

T-22	US-16/17	Goal progress bar on dashboard Progress bar widget on main dashboard: goal name, contributed/target amounts, animated fill bar; tappable to category detail.	Bar fills proportionally to contribution; 100% shows completion state; visible on dashboard without scrolling.	Lilla	1d
US-21 | Onboarding flow					
T-23	US-21	Onboarding screen flow Multi-step: (1) income & currency, (2) rule setup with presets (60/20/20, 50/30/20), (3) optional first event, (4) optional savings goal. Skip available from step 3.	New user completes onboarding in under 5 minutes; skip lands on dashboard with sensible defaults; no step requires back-navigation to proceed.	Krisztián	3d
US-22 | Deployment, Mobile & infrastructure					
T-24	US-22	Responsive mobile-first layout system Tailwind config for 375px+ primary viewport; bottom navigation bar; safe-area insets for iOS; touch targets minimum 44px.	All screens usable on 375px and 390px widths without horizontal scroll; touch targets pass 44px minimum.	Lilla	2d
T-25	DEPLOY	Docker Compose stack & Nginx config docker-compose.yml with nginx, backend, db services; Nginx routing /api/* to backend:8000; .env.example committed.	docker compose up --build starts all services; localhost serves frontend; /api/v1/docs loads; DB persists on restart.	Bálint	2d
T-26	DEPLOY	VPS deployment & CI pipeline GitHub Actions: pytest on PR to main; SSH deploy script pulling main and running docker compose up --build -d on VPS; TLS via Let's Encrypt.	Green CI required to merge; push to main triggers deploy; app accessible at production URL over HTTPS.	Bence	2d
Integration, testing & buffer					

T-27	ALL	End-to-end integration testing Full journey: register → onboard → add income → log 3 transactions → receive alert → view safe-to-spend → add event → view reminder.	All steps complete without error on staging; KPI activation loop (rule setup + 3 transactions + proactive feature) verified end-to-end.	Bence	2d
T-28	ALL	Bug fixes & polish Issues from integration testing fixed; UX polish: loading states, error messages, empty states; dashboard load performance check.	No P0/P1 bugs open; dashboard loads under 2 seconds on 4G; all empty states have helpful copy.	All	3d

3. Timeplan

The project runs for five weeks starting mid April. Week 1 prioritises the Docker Compose stack, authentication, and the rule engine, the prerequisites that unblock all other work. Frontend and backend proceed in parallel from week 2 onwards. The proactive alerting cluster (US-05, 07, 08) is scheduled for week 3 once the transaction data model is stable. Week 4 delivers the remaining user-facing features. Week 5 is reserved entirely for integration testing, bug fixing, and verification of the KPI activation loop.

Task	Week 1	Week 2	Week 3	Week 4	Week 5
T-25 Docker Compose stack	**Bálint**				
T-01 User registration API	**Bálint**				
T-02 JWT login & tokens	**Bence**				
T-03 Auth middleware	**Bence**				
T-04 Login / register UI	**Lilla**				
T-05 Rule CRUD API	**Bálint**				
T-06 Rule engine calc	**Kiara**				
T-07 Budget setup UI		**Krisztián**			
T-08 Category management UI		**Lilla**			
T-09 Transaction API		**Bálint**			
T-10 Transaction entry UI		**Krisztián**			
T-24 Mobile layout system		**Lilla**			
T-26 VPS deploy & CI		**Bence**			
T-12 Spending-pace algo			**Kiara**		
T-13 Alert job (APScheduler)			**Bence**		
T-14 Alert & projection UI			**Krisztián**		
T-15 Safe-to-spend API				**Bálint**	
T-16 Safe-to-spend widget				**Bálint**	
T-17 Event & bill API				**Kiara**	
T-18 Event reminder job				**Kiara**	
T-19 Event planner UI				**Lilla**	
T-20 Category history UI				**Krisztián**	
T-21 Savings goal API				**Kiara**	

T-22 Progress bar widget				**Lilla**	
T-23 Onboarding flow				**Krisztián**	
T-27 E2E integration test					**Bence**
T-28 Bug fixes & polish					**All**

Key scheduling dependencies

T-05/T-06 (rule API + engine) must complete before T-12 (spending-pace algorithm), which reads budget rule limits.

T-09 (transaction API) must complete before T-12 (spending-pace), which requires stored transaction data.

T-17 (event API) must complete before T-15 (safe-to-spend API), because the safe-to-spend formula subtracts upcoming event costs from the remaining budget.

T-25 (Docker Compose) is scheduled in week 1 so the staging environment is available for backend integration testing from week 2 onwards.